CONSOLIDATED SCOPE OF WORK (SoW)

Next.js Frontend, Node.js Backend, Custom CMS, VPS Hosting & 1-Year Managed AMC Services

Document Type	Consolidated Scope of Work	Document Version	v1.2 (Next.js & Node.js)
Project Scope	Next.js UI, Node.js CMS, Hosting		

1. Executive Project Overview & Objective

The objective of this engagement is to design, develop, deploy, host, and maintain a highly secure, enterprise-grade institutional platform. The system combines a cutting-edge decoupled Next.js frontend with a fast, scalable Node.js backend. This acts as the centralized engine powering a customized, robust, and perfect Content Management System (CMS). The architecture completely decouples content modification rules from delivery mechanics, eliminating raw source updates for non-technical administrators.

To protect long-term delivery benchmarks, infrastructure parameters, and maintenance guidelines, this contract outlines terms across: (i) Next.js Frontend App Design, (ii) Node.js REST API & Custom CMS Engineering, (iii) Fully Integrated VPS Server Provisioning, and (iv) 1-Year Managed Server Services & Annual Maintenance Contract (AMC).

2. Next.js Frontend Framework & Core Interface Scope

The user-facing layer will be developed using the modern Next.js React framework, utilizing advanced rendering capabilities (Server-Side Rendering / Static Site Generation) to guarantee exceptional page speeds, dynamic fluid routes, and institutional brand adherence. Key frontend requirements encompass:

- **Next.js Core Architecture:** Leveraging decoupled layouts with strict multi-breakpoint responsiveness, optimizing views for mobile interfaces, tablets, laptops, and wide screens seamlessly.
- **Modern UI/UX Branding:** Tailored layout directories constructed to fit custom institutional guidelines, focusing on optimized content layout grids, high typography clarity, and elegant interaction workflows.
- **Advanced Speed Benchmarking:** Application of native Next.js asset optimization parameters including automatic web image compression, font preloading strategies, code-splitting routes, and dynamic lazy loading.
- **SEO & Analytics Integration:** Native semantic layouts, clean URL path strategies, automated interactive sitemaps, and programmatic meta-tag fields injected dynamically on a per-page scale. Includes embed setups for Google Analytics and Google Search Console tracker scripts.

- **Compliance Standards:** Absolute validation checks ensuring full alignment with international web accessibility guidelines, recommended to satisfy WCAG 2.1 Guidelines and compliance norms.

Mandatory Interactive Layout Views & Templates:

- **Dynamic Homepage:** Equipped with rich content areas including interactive announcement modules, news crawls, hero sliders, alert popups, and real-time announcement matrices.
- **Institutional Directories:** Unified layout structures mapping out dedicated About Us hubs, categorized multi-tier Department matrices, detailed Course indices, searchable Faculty Profiles, multi-category Media Galleries, and an interactive Events Scheduler.
- **Form Intersections:** Secure communication views (Contact forms, localized Inquiry forms) configured with asynchronous submission vectors to feed structural data captures into the backend server.

3. Node.js Backend Architecture & Perfect Custom CMS

The backend infrastructure will be engineered as a performance-driven Node.js REST API layer using optimized execution runtimes to manage client requests, authentication handshakes, programmatic translations, and structured file operations.

3.1. Database Engineering & Asset Storage Strategy

- **MySQL Relational Engine:** The system will utilize MySQL, a highly stable relational database known for its transactional consistency, efficient indexing, and reliable querying capacities. It ensures secure structured storage, scalable schemas, and seamless data exchange endpoints with Node.js.
- **AWS S3 Cloud Decoupling:** All physical media components handled inside the administration workspace will bypass local application drives and stream directly into secure, highly durable AWS S3 Buckets. This maintains centralized digital asset safety and seamless file optimization across the platform.

3.2. Granular Content Management Modules

CMS Functional Block	Engineering Scope & Architecture Guidelines
Perfect Media Library	Centralized media library module engineered to manage all digital objects: handles secure async uploading, categorization, and retrieval of images, videos, and PDFs. Facilitates efficient file reuse across multiple Next.js page blocks via AWS S3 asset references, tracking groups for seamless media management.
Page Card Management	Dynamic content module managing layouts following fixed template definitions. Admins have complete control to add, remove, and reorder predefined components and cards inside structured sections. Dynamic modifications outside these finalized layout parameters will represent scope enhancements.

Unified Master Engine	Allows absolute management (Create, Read, Update, Delete - CRUD actions) of master structures. Provides reusable data matrices for repetitive components including institutional Blogs, official Academic Resources, and Product indices, executing adjustments across pages via a single master form.
Enquiry Tracking System	A secure administrative console tracking all entries harvested from frontend forms. Compiles and formats customer leads, contact submissions, and specific queries into an indexed ledger, letting internal teams seamlessly handle updates and follow-up paths.
Language Switcher API	Integration of localized translation modules utilizing the cloud-driven Google Translation API to afford real-time, automated language conversions between English, Hindi, and Marathi viewers.

3.3. Node.js Control Panel Administration & Core Workspaces

- **Role-Based Access Control (RBAC):** Strict identity enforcement matching explicit privilege parameters across Admin (full operational authority), Editor (content management capabilities), and Viewer (read-only monitoring context) ranks to dictate access across dashboards, menus, and banners.
- **WYSIWYG Rich Editor:** Integration of visual text canvas environments inside content creation views, empowering team editors to execute simple typography formatting and inline media embedding.
- **Audit Trails & File Optimizers:** Granular asset parsing engines executing automatic file adjustments upon upload, paired with structural change logging to maintain complete history files of administrative actions for leadership review.

4. Dedicated VPS Server Production Environment Configuration

The production Next.js application server and Node.js REST runtime environment will reside on an isolated Virtual Private Server (VPS) leveraging Premium SSD volumes to fulfill processing benchmarks and data access speeds:

Hosting Specification Parameter	Finalized Architecture Allocation Details
Hosting Type	Virtual Private Server (VPS) – Premium SSD (Basic Configuration)
RAM	4 GB Dedicated RAM Engine Allocation
Storage Capacity	80 GB Premium High-Performance Solid State Drive (SSD) Storage
CPU Compute	2 Dedicated vCPU Core Processors
Data Transfer Bandwidth	4 TB Data Network Allocation
Data Centre Location	India (Onshore Data Infrastructure Target Location)

Operating System (OS)	Linux Ubuntu 24.04 LTS (Long Term Support System Cover)
Database Server Engine	MySQL Relational System Instance Framework
Language Support Layer	JavaScript / TypeScript Application Processing Environment (Node.js/Next.js)
Port Network Speed	Up to 1 Gbps High-Speed Connection Uplink Bursting Layer
Web Server Runtime	Apache / Nginx Web Reverse-Proxy Infrastructure
Dedicated IP Allocation	1 Unique Dedicated Static IPv4 Public Address Assignment
Server Access Context	Root Server Identity / Secured SSH Access / Cryptographic PuTTY Keys
Web Administration Panel	CloudPanel (High-Performance Open Source Panel Management Suite)
Core Server Backup Route	Automated Weekly Full Server Bare-Metal Image Backups
SSL Security Layers	Automated Free SSL Certificate Automation and Renewal Enforcements
Server Management Class	Fully Managed Server Environment Managed continuously by Vendor Architecture

5. Enterprise Security Measures & OWASP Core Practices

The application layer will implement secure coding standards to insulate data interfaces against hazards referenced inside the Open Web Application Security Project (OWASP) criteria:

- **Injection Mitigation:** Exhaustive application of prepared statements and parameterized object mapping across Node.js query routes to isolate database connections from external string injection loops.
- **Cross-Site Scripting (XSS) Defenses:** Implementation of strict input sanitization routines and contextual output encoding, blocking unverified script inclusions or markup adjustments within the Next.js frontend pages.
- **Cross-Site Request Forgery (CSRF) Tokens:** Generation of unique cryptographic handshake tokens matching current administrative sessions to protect mutable REST API routes against external requests.
- **Access Perimeter Securing:** Enforcing two-factor authentication (2FA) models where applicable on administrative login portals, coupled with secure storage parameters protecting backend keys.

6. Next.js/Node.js Deployment & Technical Handover Requirements

6.1. Deployment Responsibilities & Post-Launch Support

The selected development group retains structural ownership over platform setup and final production staging, covering:

- Establishing cloud server provisioning(vercel), database layout staging, DNS records integration, and active SSL validation setups.
- Configuring SMTP mail exchange engines to route transactional frontend form notices reliably.
- Providing interactive training sessions to designated internal staff on utilizing the perfect custom CMS panels.
- Provisiing a post-launch support window lasting exactly one month dedicated to system bug fixes and baseline configuration updates.

6.2. Mandatory Deliverable Technical Handover Manuals

The following descriptive system engineering documents must be presented to the client as an absolute prerequisite to project closure and final milestone sign-off:

- **Folder Structure Document:** Comprehensive manual detailing the application repository architecture (both decoupled Next.js frontend project structures and Node.js backend directory hierarchies), detailing the explicit operational purpose of key directories and tracking modules.
- **API Documentation:** Detailed REST API manual defining every exposed Node.js endpoint, required HTTP headers, request bodies, status code matrices, payload structures, and valid integration examples.
- **SQL Database Schema Guide:** Detailed relational database architecture map containing a complete Entity-Relationship (ER) Diagram, comprehensive table blueprints, individual field descriptions, index constraints, and precise column data types.

> **INTELLECTUAL PROPERTY & TRANSITION RIGHTS:** *All compiled Next.js source code configurations, Node.js API logic, relational MySQL schemas, graphic vector assets, and system manuals remain the absolute property of the client. The vendor is bound to deliver complete technical cooperation and access files to enable a smooth system transition if a vendor change occurs.*

7. Annual Maintenance Contract (AMC) & Managed VPS Operations

> **AMC TERM VALIDITY:** *The Annual Maintenance Contract (AMC) and Fully Managed Server scope stands for a fixed duration of exactly one (1) year, starting on the day the completed platform goes live on the active production server environment.*

7.1. Managed Infrastructure Operations & Support Breakdown

Throughout the active 1-year contract lifecycle, the vendor will execute continuous monitoring and administrative support operations, organized below:

A. Technical Core & Operating System Support:

- Verify and double-check that automated server backups are executing and working properly.
- Proactively monitor database disk usage metrics to mitigate unexpected volume constraints.
- Maintain, update, and patch the active Web Administration Control panel (CloudPanel).
- Perform runtime health audits, apply operating system updates (Ubuntu Linux), and verify remote management tools.
- Monitor real-time server resource utilization, processor bandwidth thresholds, and overall hardware health.
- Check server logs systematically for runtime errors, software warnings, or security anomalies.
- Apply required monthly operating system security patches and firmware updates immediately.

B. Access Control, Perimeter Protection & Identity Security:

- Systematically review and manage administrative user account access privileges.
- Change root administrative credentials and specialized user passwords when required or scheduled.
- Enforce strict password complexity and systemic strength guidelines across all control points.
- Manage, configure, and update the built-in server security firewall configurations.
- Explicitly allow or restrict server SSH/Web access connections from specific IP hosts or trusted institutional vectors.

C. Application, Next.js Content & Network Support:

- Provide periodic website content updates as requested by authorized institutional users.
- Assist internal operators in uploading web banners, legal notices, media files, and official documents.
- Deliver minor configuration support for new page creation, content formatting, and element styling.
- Regularly update core application frameworks, security patches, and track network bandwidth utilization charts.
- Maintain coordination with infrastructure hosts, track domain names, and execute active SSL certificate updates.
- Execute weekly server imaging backed by structural restoration audits to ensure bulletproof disaster recovery plans.
- Ensure the Next.js pages, Node.js endpoints, MySQL records, and integrated scripts consistently run smoothly.

D. Operational Progress Logs & Compliance:

Submission of highly detailed Monthly or Quarterly Maintenance Compliance Reports documenting: (i) Framework upgrades and operating system patches executed, (ii) Backend or frontend runtime issues resolved, (iii) Core platform uptime percentage verifications, and (iv) System optimization parameters for ongoing maintenance.

7.2. AMC Handover Registries

The service provider will preserve an encrypted logbook tracking active infrastructure keys, control configurations, and access boundaries. A clear system changelog must keep record of every system patch, database optimization, or firewall adjustment, resulting in a full operational summary report submitted to the client upon contract term conclusion.